
ESTRUCTURAS DE DATOS

TP 1

Análisis de algoritmos.

1. Dado el siguiente algoritmo:

```
func buscar(lista []string, buscado string) int {  
    for i := 0; i < len(lista); i++ {  
        if lista[i] == buscado {  
            return i  
        }  
    }  
    return -1  
}
```

- a) Calcule $T(n)$ en el peor caso. ¿Cuál es el Orden del algoritmo?.
- b) ¿Podemos decir también que el algoritmo tiene $O(n^2)$? ¿Por qué?
- c) ¿Podemos decir también que el algoritmo tiene $O(\log_2 n)$? ¿Por qué?
- d) ¿Podemos decir también que el algoritmo tiene $O(1)$? ¿Por qué?

2. Si se tiene un algoritmo con complejidad computacional $O(n^2)$ y otro con $O(2^n)$, donde n es el tamaño de la entrada. ¿Cuál utilizaría usted?

3. Calcule el tiempo de ejecución para los siguientes algoritmos suponiendo que cada operación elemental tarda un tiempo constante. Luego indique el Orden en el peor caso para cada uno de ellos.

a)

```
func ejemplo1(n int) {  
    for i := 0; i < n; i++ {  
        fmt.Println(i)  
    }  
}
```

b)

```
func ejemplo2(n int, m int) {  
    for i := 0; i < n; i++ {  
        fmt.Printf("(i: %v)", i)  
    }  
  
    for j := 0; j < m; j++ {  
        fmt.Printf("(j: %v)", j)  
    }  
}
```

c)

```
func ejemplo3(n int) {  
    for i := 0; i < n; i++ {  
        for j := 0; j < n; j++ {  
            fmt.Printf("(i: %v, j: %v)", i, j)  
        }  
    }  
}
```

d)

```
func ejemplo4(n int) {  
    for i := 0; i < n; i++ {  
        for j := i; j < n; j++ {  
            fmt.Printf("(i: %v, j: %v)", i, j)  
        }  
    }  
}
```

4. Si en el ejercicio 2 el tamaño de la entrada es 3. ¿Cual utilizaría?
5. Se sabe que la cantidad de comparaciones que realiza la búsqueda binaria para determinar si un elemento se encuentra o no en un arreglo ordenado de tamaño n es de $O(\log_2 n)$. Cuantas comparaciones realizaría en el peor caso si $n=2$, $n=4$, $n=8$, $n=16$, $n=32$, $n=64$, $n=128$, $n=256$, $n=512$, $n=1024$.

Contrástelo con una búsqueda lineal. En este caso el número de comparaciones es de $O(n)$.
6. ¿Qué es mejor para usted un algoritmo con $O(n)$ o uno con $O(n/2)$? ¿Tiene sentido hablar de $O(n/2)$? Y hablar de $O(n+4)$

